Combinatorial Requirements

1. Physical and Combinatorial Requirements (scenarios and variability):

- ***PRIORITIZATION***

  A wild or substitute symbol is one that can be used in place of other symbols in the pay table combinations. Consider a game where the wild symbol W substitutes for symbols A and B, and the reels stop with symbols {W, W, A} on the payline. This matches both left 3 A and left 2 B pay rules (combinations {A,A,A} and {B,B} respectively). While we could pay both, it is more common to pay on the highest win only, hence the common rule on slot games of **"Highest win pays on each payline"**. This is also referred to as **prioritization**, as we order the priority of the pay rules, and determine which single rule to pay.

  > **Wilds pitfall**
  >
  > A wild symbol has only substitute semantics. Every time a designer puts a wild symbol in the paytable with its own payout, he breaks the mechanics and makes a synthetic mistake, since a wild by definition has a payout that is defined implicitly. It pays as any symbol from the paytable for which it can be substituted, but pays only as one of them per each win combination! If for a given predefined winning combination it results in a different payouts, then it is paid according to the prioritization we discussed above. In the corner case, when a winning combination contains only wild symbols, it pays as the highest paying symbol on the paytable - that is, the wild symbol only substitutes for the highest paying symbol.

- ***OVERFLOW***

  Let's analyze the following piece of game specification (derived from a similar feature in the game *"Golden Leprechaun: Knock! Knock! Knock!"*): In a Slot machine with a screen of 5 reels and 3 rows (5 x 3), the following feature is defined:

  "From 2 to 4 wilds appear randomly on the reels, each of which can be cloned with 0 to 4 new wilds."

  However, what will happen when we maximize all of these feature parameters? The screen can only hold 15 tiles, and maximizing the feature parameters will result in 1 more tile above

the maximum. Where will it be placed? Although the solution to this case may be trivial, due to the semantics of the wild symbol, it must be considered and described, since much of the text in the specification can serve as the basis for the texts in the game's help, with which the player will become familiar. Here are a few solutions to further describe the feature:

- Re-adjusting the parameters, the number of random wilds goes from 1 to 3, and the number of new clones from 0 to 5.
- Wilds can overlap, and if a wild lands on a wild that is already on the screen, the multiplier of that wild increases with x1, as the initial multiplier of the wild is x1. Each multiplier is applied only to the line in which it participates, and if there is more than one wild with a multiplier greater than x1 - the multipliers values of those wilds (with > x1) are aggregated with addition.

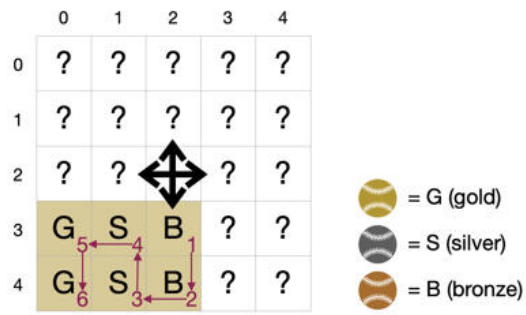
But what if the symbols that were cloned were scatters. Scatters are special symbols which can have its own paytable, its own pay mechanics or can even lead to events with a predefined payout method or functionality. Usually, scatters are blockers for the normal payout strategy. Then the solution to the not well-defined event is not so trivial, since for 16 scatters we may have a unique event defined, but it will actually be impossible. Naturally, there are radical solutions, such as making scatters wilds at the same time, but will this fit well into the game, and will it inject new problems? We do not know since the game we are observing is abstract. Such cases should be solved at the design level, imagined mentally by the designers, and such decisions should be made that favor the game feeling, and not inject synthetic mechanics that negatively affect the entire game.

- ***VARIABILITY***

Let's analyze the following piece of game specification (derived from a similar feature in the game *"FanDuel: Classic Invaders"*): In a Slot machine the following feature is defined:

"When the feature is executed, a new scene is revealed where there is a grid/board of cells/tiles with size 5 x 5 and the traversal of the grid starts from the center position. On each step we can visit only a tile that was not visited before and has a common side with the currently visited tile. The grid contains exactly 24 symbols which are hidden and revealed after they are visited (plus 1 for the start position which is visited to make a total of 25 positions). The symbols are exactly 3 types: Gold, Silver and Bronze and each type has a payout. The payout is made when the feature ends and the feature ends when for the first time in the path of the traversal - three symbols of one type are collected (revealing the board) and the corresponding prize for that type is awarded."

So where are the problems of that feature? Lets label the three type of balls, i.e. bronze = "B", silver = "S" and gold = "G". And lets have the following traversal:



From the pigeonhole principle, all paths with size 7 will contain at least 3 type of symbols of the same type, which is the feature "end condition" and respectively "win condition". If the path is of length less than 6, then the first time when 3 types of symbols are collected/revealed along the path - the feature end/win condition is met. But what about the path from the above grid? We have a path of length 6 and we are on a dead-end cell, meaning that by the given conditions for the traversal on the grid - we cannot continue, and also we haven't collected 3 types of symbols and we can't win anything. So what happens here?

We need to either populate the board so that in all these 8 paths of length 6 have at least 3 identical symbols on them or to change the rules of the traversal. Yes, but if we do that and come up with an algorithm that does it, it will not be sufficiently nondeterministic because by regulation we will need to reveal all the grid cells after the bonus ends and there must have to be at least one path with a gold outcome and at least one path with a silver outcome. Then the probability of winning a golden path becomes no less than 1/8 and can be intercepted by the player which may be too restrictive *(that is because the algorithm must include counting all of the paths and selecting from them in order not to include backtrack for validating the board to have either one of each path type on it)*. OK, so we decided now to change the rules of the traversal so that we can traverse from one cell to another if: **the two cells share a common side or are located at both ends of a row or column on the grid (and the newly visited cell is unvisited)**. The only thing left here is to validate that the board is populated with at least one of all possible winning paths, and this can be done by completely randomized probability-based population for each symbol and each cell and a backtracking algorithm until the backtrack validates the board.

But we are not done, yet. Most likely, the designer will want some RTP set aside for this feature, for example lets say 30% of the game's total RTP, and may also want this bonus to be available for purchase (Bonus Buy). But the game has exactly 3 exits, which firstly makes it difficult to configure to fit RTP frames (since we cannot reduce the symbols of one type, because any imbalance will cause the backtrack algorithm to return an invalid board and this can slow down the process significantly), and secondly the feature is with too few exits to be interesting to be purchased at all. If we buy the feature we will know that we will win either gold, silver or bronze path, i.e. only 1 of total 3 payouts. **Again variability shows its key importance**. This problem can be solved by randomly selecting the winnings for each path from lets say 3 possibilities before starting the traversal of the board. For example, for path with gold win we will have one selection out of {x200, x500 or x1000} stake multiplier, for silver we will have one selection out of {x20, x50 or x100} and for bronze we will have one selection out of {x2, x5 or x10}. This way we will have greater randomization of the possible outcomes and we will be able to WEIGHT the chances of the selection of winning outcomes for each type of path and thus we will be able to configure the feature to fit into each RTP target interval.

- ***DESCRIPTORS vs. PHYSICAL PLACING:***

  This case study is a special case of variability, but it is the most extensive topic, which I will try to summarize in a few short paragraphs.Let's analyze the following piece of game specification: In a Slot machine the following abstract feature is defined:

  "The game must have special symbols with stacks [x, ?] (i.e. when the symbol appears, it must be in a stack with at least x symbols of the same type) and the hit rate of those special symbols must be Y for some Y < 1."

  This is a very common requirement that is set explicitly or implicitly in the specification and in almost every game we have to comply with it. But what are the stumbling blocks here? If we try to place these physical symbols on the reels, it would result in too much manual adjustment, which is slow and is prone to bugs. We also need to think about how these special symbols can appear in any context. Imagine if we physically placed them on the reels so that they could never appear together (in combination) with any other special symbols or case, which would lead to a loophole in the game that would be hard to detect since the reels could be more than 10 and even reach 50-100 in some games. Therefore, in order to achieve the full and much-needed **variability** that will lead to a better game feeling, it is necessary to have all symbols with descriptors that describe where on the reel they are located and actually not having special symbols physically located on the reels. This, although it solves

the variability problem, injects new problems that are insignificant in terms of complexity to solve compared to the one they solved. Some of them concern whether the special symbol can be placed on top of another special symbol, how it adapts if it can't, and whether it acts as a blocker if it lands on a winning combination, but we won't address those issue here (we will just say that they are solvable). What we will mention here is that the information of the descriptors that describe where on the reels the special symbols are distributed is created **dynamically** each time, making it as configurable as possible and thus all requirements for frequency of occurrence can be met. Also, an additional information can be added to the descriptor about how stacked the symbol is, whether it is on the screen, and other necessary information. Through this strategy, we can populate the reels in such a way that it is richer in special symbols off-screen and poorer on-screen, which will increase NEAR MISS, but it can also make ANTICIPATION too high, which is not desirable, but with great power come great responsibilities and risks. The point is that with this option with descriptors we have high configurability and variability and that is the main goal we are pursuing. Everything else is accumulated with experience.

special symbols

Every time some special symbol must be distributed across the reels, it is good to think about whether they should be on the reels or whether it will have the same effect if they are not visualized at the front-end on the reels when they are spinning, but placed randomly or based on some other logic on the screen, after the reels have stopped or before they have spun but not during the spin. THIS IS OF KEY IMPORTANCE, which can lead to much easier mechanics to develop, and in some cases even to a better game feeling.